

W03 Assignment: Rafting Site – About Us Page

Overview

The assignment this week introduces a multi-page website project about a white water rafting company that you will work on for the next few weeks in this course. You will begin by building one of the pages on the site, "**About Us**". This page provides an overview of the purpose of the website, history, and other promotions for the company.

Instructions

Step 1: Introduction – The Rafting Site

The rafting site assignment is a multi-week project. You will start developing the site by building one of the pages, the **About Us** page. This website represents a fictional, white water rafting company.

1. Read the [rafting site description](#) to get an overview of the project and to see examples of real companies that do this type of business.

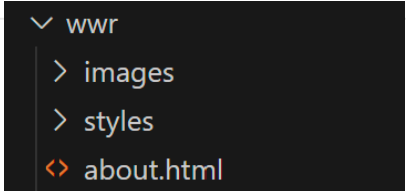
Step 2: Folder and File Setup

1. In VS Code, be sure you have your **wdd130** directory open and **add** a sub-folder named "**wwr**" (wwr stands for "**w**hite **w**ater **r**afting"). This folder will contain all the pages and assets required for the website.
2. Add two subfolders *within* the **wwr** folder named "**images**" and "**styles**".

Do not be confused that you already have root level (**wdd130**) folders named *images* and *styles*. These new folders are specifically for your rafting website project and are contained within the **wwr** project sub folder.

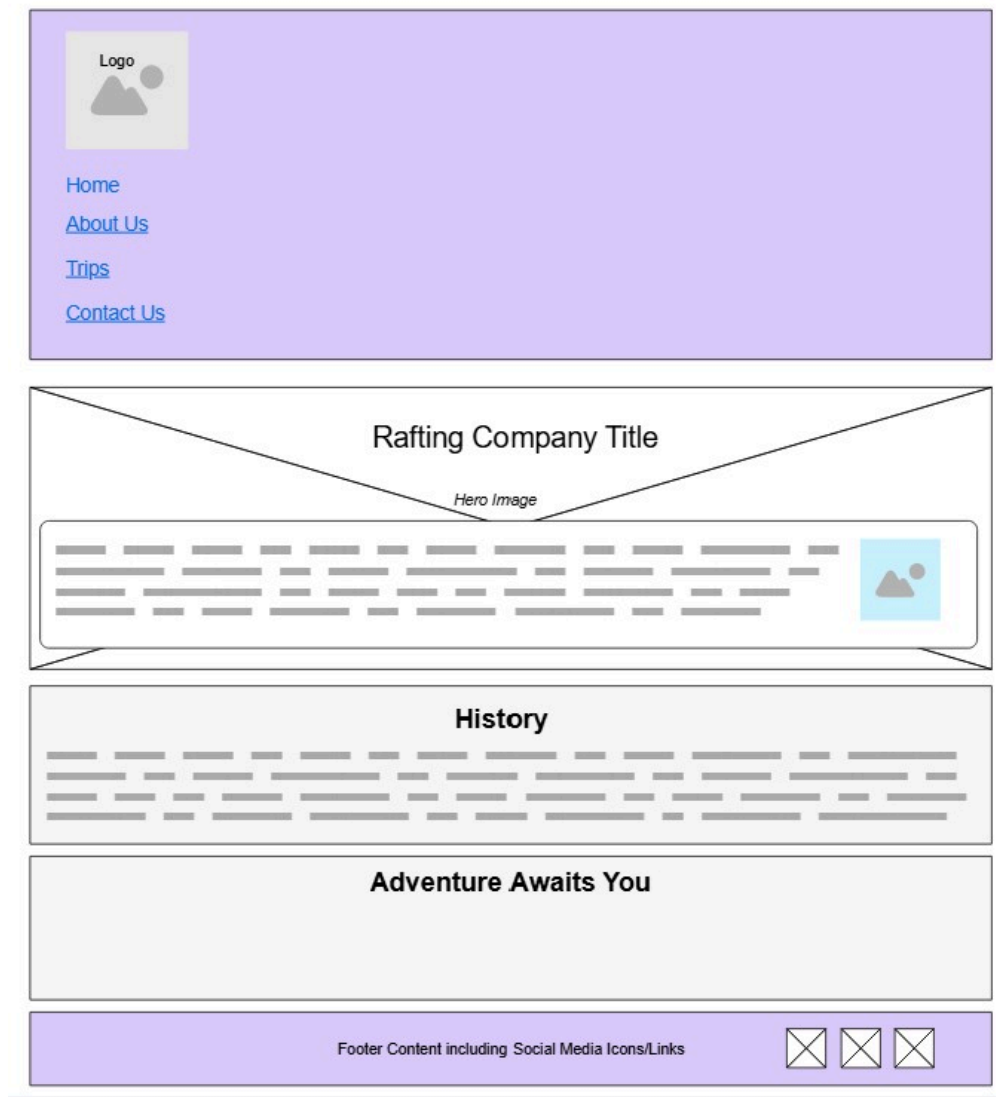
3. Add a file named "**about.html**" in the **wwr** directory.

Step 3: Develop the Page Structure: HTML



```
▼ wwr
  > images
  > styles
  <> about.html
```

Use the following wireframe/sketch design as a guide to develop your page structure, layout, and style for this about.html page.



1. In your **about.html** document, begin writing the necessary HTML structure by starting with the **head** with its standard, required **meta** and **title** content.
2. Add a **meta description** to the **head** with concise content that is an enticing summary of what the web page is about.
3. Add a **meta author** element with the content being your full name.
4. In the **body** parent tag, add the following elements: **header**, **main**, and **footer**.
5. The **header** contains two (2) items:
 1. A rafting site **logo** that you have created or that you have downloaded from the choices provided at [WWR Sample Logos](#)

► Check Your Understanding

2. A **nav** element with four (4) child **<a>** tags with associated text content. The **href** attribute values, the page references, can be set now even though the

pages do not all exist yet.

- ✓ index.html (the future home page of wwr, not to be confused with the course home page.)
- ✓ about.html
- ✓ trips.html
- ✓ contact.html

► Check Your Understanding

6. The **main** element should contain the following three (3) elements:

1. A **div** with a class of **hero** that contains these items:

- ✓ A hero image **img** that will be in the entire background of the **div** background.

Be sure to pick an image that is large enough in dimensions to cover the entire **div** background.

Help: Image Resources

- [Course Rafting Image Repository](#)
- [Web Frontend Resources](#) – see Images and Graphics section.

All images used on your site must be optimized which means not pixelated and less than or equal to 125kB in size.

- ✓ A **h1** element that has the title of the rafting company as the text content.
- ✓ An **article** element that contains:
 - A smaller **img** element that portrays a happy client or happy, working employee.
 - A **p** element with the company purpose, mission, creed, motto, etc. It is OK to use non-sense, placeholder language for now.

2. A **section** element with the following contents:

- ✓ A **h2** element with a section titled "**History**".
- ✓ A **p** element with a brief history of the company. It is OK to use nonsense language.

3. A **section** element with the following contents.

- ✓ A **h2** element with a section titled "**Adventure Awaits You!**".

In the next assignment, you will add images to this section. For now, this section content is empty.

► Check Your Understanding

7. A **footer** element that contains:

- ✓ A **p** paragraph element containing a copyright symbol, the year, the rafting company name, and your name.
- ✓ A **nav** element with a class assignment of **socialmedia** that contains three (3) **img** elements that are encased by **a** anchor elements pointing to three different social media outlets. The links can be generic.

► Example

Social Media Icon Resources

[iconfinder.com](#)

[Google Icons](#)

Example

Here is an example of what the page looks like rendered in a browser to this point:



- [Home](#)
- [About Us](#)
- [Trips](#)
- [Contact Us](#)



About Us — Exciting Expeditions



Here at the Exciting Expedition, our whitewater rafting trips are designed to provide thrill-seekers with an unforgettable experience on the water. Our expert guides ensure safety while you navigate through stunning landscapes and challenging rapids. Oars Up!

History

Exciting Expeditions was founded in 1995 by a group of adventure enthusiasts who wanted to share their love for the great outdoors. Starting with just a handful of rafts and a passion for exploration, the company has grown into a leading provider of whitewater rafting experiences.

Adventure Awaits You!

© 2026 Exciting Expeditions Xavier Rodriguez



Note that this is just an example of the page rendered in the browser with HTML only and valid images.

Step 4: Write the CSS

1. Create a CSS file for your rafting site named "**rafting.css**" and store it in the **styles** folder within the **wwr** folder.
2. Link the CSS file to your **about.html** page using the **<link>** element in the **<head>** of your HTML document.

► Check Your Understanding

3. Define CSS variables in the document **:root** pseudo-class selector at the top of your **rafting.css** file by using a color scheme of your choice. This means choose colors that work well together providing enough contrast when used together.

▼ Check Your Understanding

The following is just an example. Use your own colors and fonts that you selected and referenced.

```
:root {
  --primary-color: #055a05; /* forest green */
  --secondary-color: #f4a261; /* warm sand */
  --accent1-color: #264653; /* dark slate blue */
  --accent2-color: #e9c46a; /* light golden yellow */
  --heading-font: "Times New Roman", "Georgia", serif;
  --body-font: Arial, Helvetica, sans-serif;
}
```

The fonts used in this example are classic serif and san-serif fonts that provide a formal and traditional look, they are generally available to all modern browsers.

You will use these variables in your CSS rules.

For example, to set the header background color to use the **primary-color** and the foreground color (font color) to **--accent2-color**:

```
header {
  background-color: var(--primary-color);
  color: var(--accent2-color);
}
```

4. Reset the CSS box-model properties for all elements to zero (0) using the universal selector and these declarations:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

5. Style the entire document **body** text content to use the **body-font** CSS variable. Also add a CSS declaration to set the text color for the content.

```
body {  
  font-family: var(--body-font);  
  color: var(--accent1-color);  
}
```

6. Limit the page width by combining the **header**, **main**, and **footer** elements and center the content on the display using an auto left-right margin.

```
header, main, footer {  
  width: 840px;  
  margin: 0 auto;  
}
```

This CSS selector **header, main, footer {** targets three different HTML elements simultaneously by using a comma to specify each of these child elements of the document body.

7. Style the headings (all or just those you are using) by declaring the **font-family** and text **color**.

```
h1, h2 {  
  font-family: var(--heading-font);  
  color: var(--primary-color);  
}
```

8. Style some basic, consistent **margin** and **padding** declarations for the **nav**, paragraph, **section**, and **article** elements.

```
nav, p, section, article {  
  margin: 16px;  
  padding: 8px;  
}
```

9. Remove the underlines from the social media image links in the **footer**.
This default underline on hyperlinks is considered to be visually distracting and unnecessary when applied to images as links.

```
.socialmedia a {  
  text-decoration: none;  
}
```

The **descendant selector** (`.socialmedia a`) only applies to anchor tags (`a`) that are children of elements with the `socialmedia` class and not to all anchor tags.

Step 5: Style the Hero Div with Overlays

The following image is an **example** of the overlay.



Example of the hero div with a heading and article overlay

1. Style the **div** with the class of **hero** with a **relative** value so that its children elements **h1** and **article** can be positioned over the top **absolute** within that **div**.

► Check Your Understanding

2. Style the image within the **div class="hero"** to fill the entire width of the **div** and to be a block element. This is important because the image is the background of the hero div and it needs to fill the entire width of the div.

```
.hero img {  
  display: block;  
  width: 100%;  
  height: auto;  
}
```

The image should be large enough to fill the space without being pixelated or distorted because its intrinsic size is too small to fill the space.

We use height auto in order to maintain the aspect ratio of the image based upon the given width, in this case, 100% of the container.

3. Style the **h1** heading using a **position** that is **absolute** and decide where to position over the image so that it fits. The example given above uses the following CSS declarations:

```
h1 {  
  position: absolute;  
  top: 20px;  
  width: 100%;  
  padding: 16px;  
  text-align: center;  
  opacity: 0.7;  
  background-color: var(--accent1-color);  
  color: #fff; /* white text - make sure there is enough contrast with t  
}
```



4. Style the **article** using absolute positioning and a style of your choice.
5. Style the **img** inside the article by floating it to the right and providing some margin and a specific width. The CSS for this image is important as we need to be specific to only this image on the page. The example above uses the following CSS:

```
.hero article img {  
  float: right;  
  width: 125px;  
}
```

You may wish to add other styling to the image such as a **border**, **margin**, **box-shadow**, etc., depending upon your image and design.

► [Help: Example Page Screenshot](#)

Step 6: Testing

1. Test your work continuously as you work through the steps by having your page loaded in your local browser using **Live Server** or **Five Server**.

You do **not** need to be connected to the internet while using a local server to test your pages locally during development.

2. Commit your changes and sync them to your wdd130 GitHub Pages enabled repository.

3. Audit your page using the ✓ [Page Audit](#) tool to verify that you have the basic document content.
4. Make corrections as needed and be sure to recommit and sync your updated work as needed.

Submission

1. Submit your GitHub Pages enabled URL address in Canvas. Here is an **example** of the URL you will submit.

`https://yourgithubusername.github.io/wdd130/wwr/about.html`